

Figure 3: **Token co-occurrence reinforcement.** Even if only one token repeats in context, the self-reinforcement loop triggers. “..X..Y..” denotes 2 tokens are kept unchanged. The mean and variance are computed over 1,000 randomly generated samples.

$\mathcal{M}(w[s^1; s^2; \dots; s^{n-1}; w_1 \dots w_{i-1}])$, where s is the repeating sentence, n denotes the number of occurrences, and w_i is the i -th token in the sentence s . Previous research finds the self-reinforcement effect, where the probability of generating the sentence s of length L_s occurred N times in context, $\text{TP}_N = \frac{1}{L_s} \sum_i \mathcal{P}_{\text{REP}}(w|w = w_i; n = N)$, almost monotonically increases with the number of N . While the generation of repetitive sentences above can be understood as the influence of a single surface feature, we investigate more sophisticated surface patterns, which can be causes to behaviors of in-context learning.

3 SELF-REINFORCED SURFACE FEATURES FOR IN-CONTEXT LEARNING

In accordance with Figure 1, we set $s = [\mathbf{F}_I; x_1; \mathbf{F}_O; y_1]$ and have,

$$\begin{aligned} \mathcal{P}_{\text{REP}}(w|n = K) &= \mathcal{M}(y|\overbrace{(\mathbf{F}_I, x_1, \mathbf{F}_O, y_1, \dots, \mathbf{F}_I, x_1, \mathbf{F}_O, y_1, \mathbf{F}_I, x_1, \mathbf{F}_O)}^{K \text{ times}}). \\ \mathcal{P}_{\text{ICL}}(y|x, k = K) &= \mathcal{M}(y|(\mathbf{F}_I, x_1, \mathbf{F}_O, y_1, \dots, \mathbf{F}_I, x_K, \mathbf{F}_O, y_K, \mathbf{F}_I, x, \mathbf{F}_O)). \end{aligned}$$

Comparing $\mathcal{P}_{\text{REP}}(w)$ to $\mathcal{P}_{\text{ICL}}(y)$, we find: (1) $\mathbf{F}_I$ and $\mathbf{F}_O$ are both repeated across demonstrations; (2) In repetitive generation, x_1 and y_1 are repeated, while in ICL, x and y are changing.

To investigate the correlation between surface patterns and the resulting answer y , we gradually expand self-reinforcement patterns toward in-context learning. We achieve this by introducing random perturbations to each demonstration, imitating the role of changing x while leaving certain components, such as $\mathbf{F}_O$ and y , unchanged.

The experiments in this section are conducted over the dataset of randomly generated sentences as in Section 2 and with the four LLaMA models. The results on Wikitext-103 and BookCorpus, and results with OPT and various other LLMs can be found in Appendix D. For each experiment, we repeat the pattern 20 times and report the mean and standard deviation of the probabilities for the kept tokens.

3.1 ANY TWO TOKENS FORM A TOKEN REINFORCED LOOP

We assume that tokens are the base unit of self-reinforcement. By assessing the self-reinforcement effect among tokens, we understand the self-reinforcement effect from first principles.

Formally, given a sentence $s = (w_1, \dots, w_{L_s})$ from a corpus D , we construct a binary mask sequence

$$\hat{m} = (\hat{m}_1, \hat{m}_2, \dots, \hat{m}_{L_s}), \text{ and we define a replacement operation } \mathbf{R}(w, \hat{m}) = \begin{cases} w_r, & \text{if } \hat{m} = 0 \\ w, & \text{if } \hat{m} = 1 \end{cases} \text{ that}$$

replaces w with a randomly sampled token w_r from the vocabulary if $\hat{m} = 0$ and keep it unchanged when $\hat{m} = 1$. Note that w_r is independently sampled for each sentence and each position. As for mask sequence $\hat{m}$, we randomly sample positions to put the 0s and 1s of the mask sequence $\hat{m}$ and control the number of kept tokens. In this way, a sentence s is transformed into $s^n = (\mathbf{R}(w_1, \hat{m}_1), \dots, \mathbf{R}(w_{L_s}, \hat{m}_{L_s}))$, where $\sum_{l \in [1, L_s]} \hat{m}_l = L_t$. Then, we report the average token probability $\hat{\text{TP}}_N$ as in the previous section. Suppose we have a sentence $s = (\text{Apple}, \text{there}, \text{is}, \text{red})$

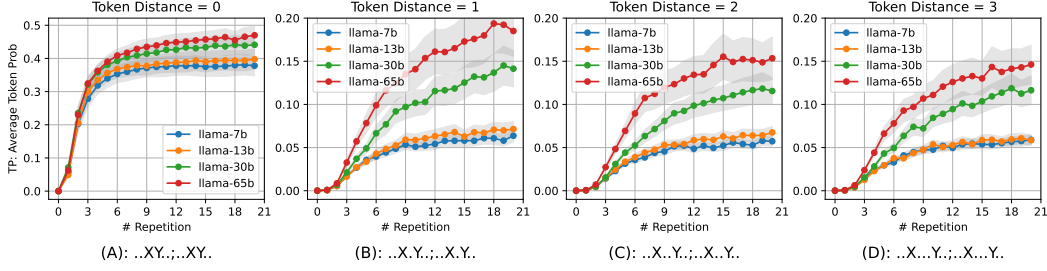


Figure 4: **Successive and distant reinforcement.** The self-reinforcement effect is the strongest when two tokens are successive, i.e., distance=0. Otherwise, the reinforcement is smaller and appears insensitive to the distance. “..X.Y..” denotes the distance between two tokens is 1. and $m = (1, 0, 1, 1)$, and the target token $w = \text{red}$. Then, the demonstrations in this section will be like ‘Apple there is red // Apple logo is red // Apple juice is red’.

Number of Tokens As depicted in Figure 3, even only one single token shared across demonstrations elicits self-reinforcement. We are particularly interested in the scenario with two tokens kept unchanged, as it reveals a fundamental rule of one token triggers the generation of the other one. We find that *the connection between any two tokens gets reinforced and the probability increases monotonically with the number of their contextual co-occurrences*. We refer to this base effect as the token co-occurrence reinforcement. When we increase the number of preserved tokens from 2 to 4, we observe a strengthening of the reinforcement effect. This is because each former token forms a reinforced connection with all the latter ones.

Distance In-Between We further examine the role of distance in token reinforcement. This analysis is confined to two tokens. Figure 4 distinctly differentiates between successive tokens (distance= 0) and distant tokens (distance ≥ 1), which we term as *successive reinforcement* and *distant reinforcement*, respectively. The successive reinforcement significantly elevates the probability, from 0 to 0.4, with only several repetitions. Conversely, the distant reinforcement provides a moderate boost to the probability, from 0 to 0.2, and appears to be indifferent to the distance between tokens.

Across all experiments, we observe a marked increase in reinforcement as model size scales, especially in the distant token reinforcement. This consistent escalation suggests that larger LLMs are more capable of following complex patterns in in-context demonstrations, which is consistent with results from Wei et al. (2023). We provide more supporting evidence of token reinforcement in Appendix D.

The link we found between any two tokens forms the foundation of sentence-level self-reinforcement. Each token is strengthened by the ones before it. In ICL, common elements in demonstrations, such as pattern words, form connections with label words like “A, B, C, D”.

3.2 REASON BEHIND TOKEN REINFORCEMENT

Token reinforcement is inherently embedded within the pre-training corpus. We find that token reinforcement could be a result of the model’s effort to maximize the likelihood of the training data.

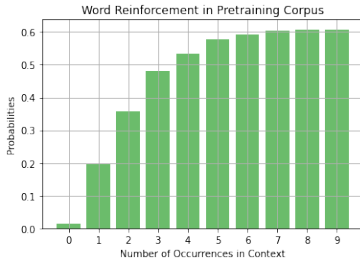


Figure 5: The probabilities of next occurrence after several occurrence observed in context.

Given a pre-training corpus D_{train} , the language modeling task over a sequence of tokens $\{w_1, w_2, \dots, w_T\}$ with length T is defined by $\max - \log P(w_i | w_{<i})$. We compute the following empirical probabilities over the corpus:

$$P_n = \mathbb{E}_{w \in V} \left[\sum_i^T P(w_i = w | \text{Count}(w, w_{<i}) = n) \right], \forall n,$$

where $\text{Count}(\cdot, \cdot)$ denotes the function to count the number of a word w in the partial sequence $w_{<i}$, and V is the vocabulary.

Intuitively, these probabilities reveal whether a particular word w is likely to recur if there have already been n instances of w

observed in the context, resonating with our analysis of token reinforcement.

We compute these probabilities over a commonly used pretraining corpus, wikipedia-english-2022³, encompassing over 5.7B tokens, using the LLaMA tokenizer to first tokenize corpus and preprocess each context window with $T = 1024$ tokens.

Figure 5 demonstrates our results. We find that the trend of word probabilities accords to our results in Section 3.1. The more instances seen, the greater the probability of the same word recurring. Therefore, we infer that the token co-occurrence reinforcement stems from the LLMs’ optimisation of the training likelihood. The LLMs manage to learn this inherent feature from the training data and generalize it to longer phrases and distant connections. This also elucidates the scaling with reinforcement, where larger models more effectively capture this feature from the training corpus.

3.3 UNDERSTANDING ICL VIA REINFORCEMENTS

Token reinforcement effect discussed in previous sections provides a new perspective to understand in-context learning. Consider the following example with three demonstrations [A,B,C,D ; A,b,C,D ; a,B,C,D ; A,b,C,(?)]. Our target is ‘D’. In this example, several reinforced connections exist concurrently. ‘A->D’ is reinforced twice; ‘b->D’ is reinforced once; ‘C->D’ is reinforced three times. Here, all three reinforcements reach a consensus and predict ‘D’ in cooperation.

However, this ideal scenario is not always the case. Consider another example [A,B,C,D ; A,b,C,E ; a,B,C,F ; A,b,C,(?)]. At this time, ‘A->D’ is reinforced once; ‘b->E’ is reinforced once; ‘a->F’ is reinforced once and et cetera. These reinforcements create conflicts and compete against each other. Thus, instead of the traditional view of ICL as a mapping from input to output, we view ICL from a feature angle, as a combination of connection of tokens, even though some of them are highly reinforced and some of them are not. The next section shows how the reinforcements play a crucial role in in-context learning.

4 THE EFFECTS OF SURFACE PATTERNS TO IN-CONTEXT LEARNING

We quantitatively study how self-reinforced surface patterns lead to both beneficial functions and detrimental effects in ICL. The experiments in this section are conducted over MMLU (Hendrycks et al., 2021) and GSM8K (Cobbe et al., 2021). Due to limit of computing resources, we randomly sampled 20 samples for each of the 57 tasks of MMLU, resulting in a collection of 1140 test samples. The demonstrations are independently drawn for each of the test samples. All experiments are conducted across three random seeds. For further experimental details, see Appendix B.

4.1 BENEFICIAL EFFECTS

Constraining Output Space. An important advantage brought by the reinforced effect is that it helps constrain the output space — with several demonstrations, connections between formatting tokens (‘Input:’ and ‘Answer:’) and label words in each demonstration (‘ABCD’) are reinforced, through distant and successive reinforcement, respectively. In this way, the LLM learns to predict either one of ‘ABCD’ as the final answer, instead of continuation sequences such as ‘Oh, an interesting question. I hope I know the answer.’.

We verify this advantage with the MMLU dataset, which is widely used to evaluate the language understanding of real-world large language models. To isolate the effect of self-reinforcement, we construct masked demonstrations for analyses. An example of how we mask demonstrations is shown in the left part of Figure 6. Particularly, a demonstration in the MMLU dataset can be divided into five parts: question content, option name (e.g., ‘A.’), option content (e.g., ‘114, 27.35’), answer indicator (e.g., ‘Answer:’) and final answer (e.g., ‘D’). Based on token reinforcement, we hypothesize that the option names, i.e., “A.”, “B.”, reinforce outputting “A,B,C,D” via distant reinforcement. The answer indicator, i.e., “Answer: ”, reinforces outputting within label space via successive reinforcement. To validate our hypothesis, we first mask all the questions and option contents in all demonstrations and keep the formatting words, final answer, and test query unchanged. Then, we

³<https://huggingface.co/datasets/olm/olm-wikipedia-20221220>